

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?

Middleware provides a structured way to process requests before they reach the final route handler. Advantages include:

- Code reusability
- Each middleware handles a specific responsibility
- Improved maintainability
- Middleware can modify requests or responses before continuing to the next function.
- Better scalability

2. How does separating middleware into dedicated files improve the maintainability of your code?

Placing middleware into separate files improves organization and readability. Benefits include:

- Cleaner route files
- Easier
- Simpler collaboration
- Higher reusability
- Better scalability

3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?

I would create a layered middleware system:

1. **Authentication middleware**
Verifies the user identity using JWT or sessions.
2. **Authorization middleware**
Checks whether the user has permission to access a route.

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., minCredits=4 and maxCredits=3)?

The backend should validate query parameters before processing them.

5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?

Possible improvements include:

- Case-insensitive matching
- Partial matching
- Fuzzy search algorithms
- Autocomplete suggestions
- Standardized filter values

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., `?token=xyz123`)? What alternatives would be more secure?

Using query parameters for authentication is insecure because:

- Tokens may appear in browser history
- Tokens can be logged in server logs
- URLs may be shared accidentally
- Tokens can leak through referrer headers

More secure alternatives include:

- JWT tokens in Authorization headers
- Session-based authentication
- OAuth 2.0
- Secure HTTP-only cookies

7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

Validation and sanitization protect the application from malicious or invalid data.

Reasons include:

- Preventing injection attacks
- Avoiding application crashes
- Ensuring correct data types
- Improving data consistency
- Protecting databases and APIs

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?

Yes. Middleware such as logging, authentication, validation, and error handling are reusable across many Express applications.

To package them:

1. Store middleware in separate modules
2. Export them using `module.exports`
3. Publish as an npm package if needed

Documentation should include:

- Purpose of the middleware
- Required parameters
- Example usage
- Expected request/response behavior
- Error handling details

9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

I would design the filtering system dynamically instead of hardcoding each filter.

Bonus – Real-World Thinking

10. How would this API behave under high traffic? What improvements would you need to make for production readiness (e.g., rate limiting, caching)?

Under high traffic, the API may experience:

- Slow response times
- Increased server load
- Memory usage issues
- Possible downtime

Production improvements would include:

Performance Improvements

- Caching with Redis
- Database indexing
- Load balancing
- Compression middleware

Security Improvements

- Rate limiting to prevent abuse
- Helmet.js for HTTP security headers
- Input validation and sanitization
- HTTPS enforcement

Reliability Improvements

- Centralized error handling
- Logging and monitoring
- Automated backups
- Health checks